simple_linear_regression

June 4, 2026

0.0.1 Simple Linear Regression

[1]:
```
# import required libraries
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

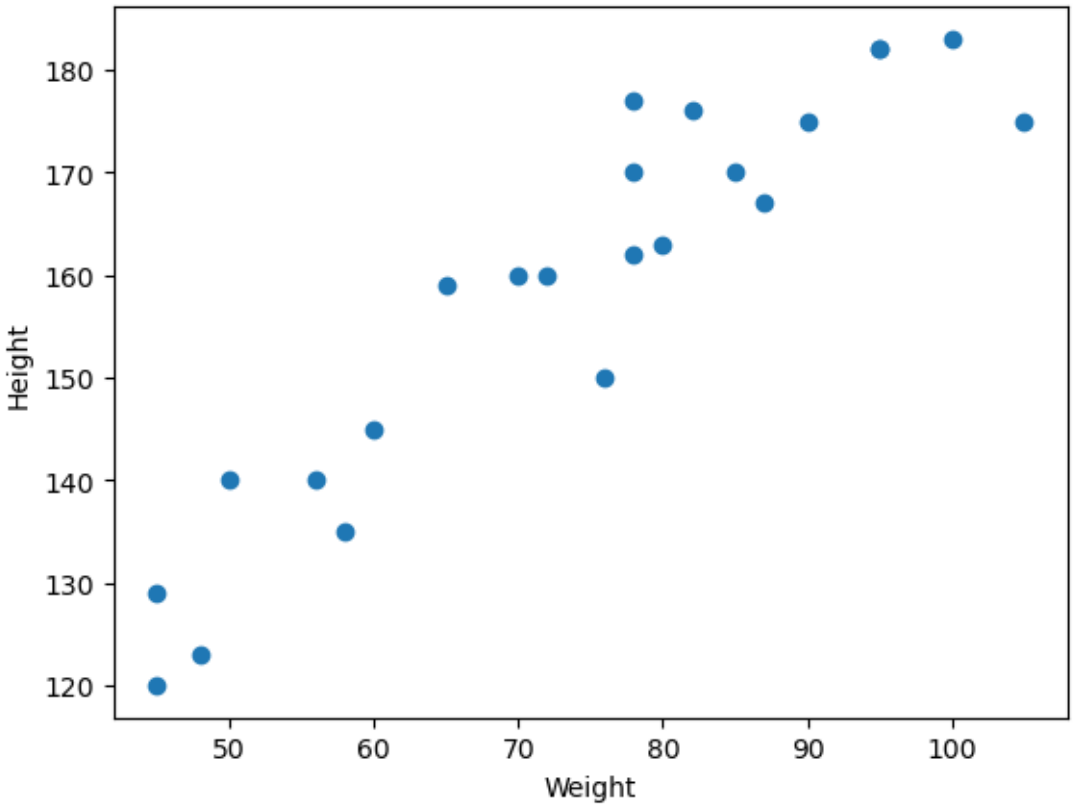
[2]:
```
# read csv data and load into variable
df = pd.read_csv('height_weight.csv')
df.head()
```

[2]:

	Weight	Height
0	45	120
1	58	135
2	48	123
3	60	145
4	70	160

[3]:
```
# Scatter Plot
plt.scatter(df['Weight'], df['Height'])
plt.xlabel("Weight")
plt.ylabel("Height")
```

[3]:
```
Text(0, 0.5, 'Height')
```



[4]:
```
# Correlation insights
df.corr()
```

[4]:

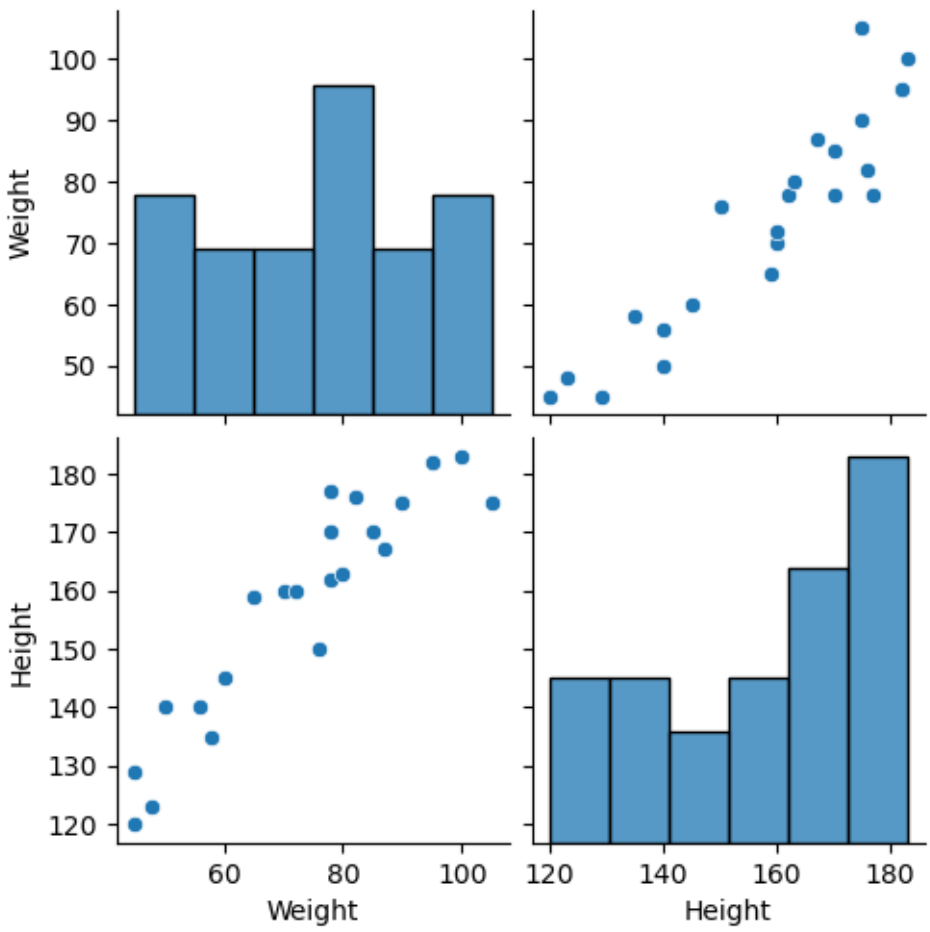
	Weight	Height
Weight	1.000000	0.931142
Height	0.931142	1.000000

Weight & Height are positively correlated

[5]:
```
# Seaborn for visualization
import seaborn as sns

sns.pairplot(df)
```

[5]:
```
<seaborn.axisgrid.PairGrid at 0x799516613e00>
```



[11]:
```
# Independent and Dependent features
# # Independent feature should be in data frame
X = df[ ['Weight'] ]

# # Dependent feature can be in series or 1D array
y = df['Height']
X.head()
```

[11]:

	Weight
0	45
1	58
2	48
3	60
4	70

[7]:
```
X_series = df['Weight']
np.array(X_series).shape
```

[7]:
```
(23,)
```

[10]:
```
y
```

[10]:
```
0     120
1     135
2     123
3     145
4     160
5     162
6     163
7     175
8     182
9     170
10    176
11    182
12    175
13    183
14    170
15    177
16    140
17    159
18    150
19    167
20    129
21    140
22    160
Name: Height, dtype: int64
```

[12]:
```
# Train Test Split
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,␣
↪random_state=42)
```

[13]:
```
X_train.shape
```

[13]:
```
(17, 1)
```

(17, 1) means 17 rows and 1 feature

0.0.2 Standardization Process

[14]:
```
from sklearn.preprocessing import StandardScaler
```

[17]:
```
scaler = StandardScaler()
```

```
# Actual formula is Z-score here. Each data point minus mean divided by␣
 ↪standard deviation.
X_train = scaler.fit_transform(X_train)
```

[20]:
```
X_train
```

[20]:
```
array([[-0.87662801],
       [ 1.66773133],
       [ 0.33497168],
       [-1.48242785],
       [ 1.36483141],
       [-1.6641678 ],
       [-0.75546804],
       [-0.1496682 ],
       [ 0.21381171],
       [-1.36126788],
       [-0.99778797],
       [-0.02850823],
       [ 1.06193149],
       [ 0.57729161],
       [ 0.75903157],
       [ 0.88019153],
       [ 0.45613165]])
```

[18]:
```
X_test = scaler.transform(X_test)
```

[19]:
```
X_test
```

[19]:
```
array([[ 0.33497168],
       [ 0.33497168],
       [-1.6641678 ],
       [ 1.36483141],
       [-0.45256812],
       [ 1.97063125]])
```

[21]:
```
# Apply linear regression
from sklearn.linear_model import LinearRegression
```

[22]:
```
regression = LinearRegression()
```

[23]:
```
regression.fit(X_train, y_train)
```

[23]:
```
LinearRegression()
```

[25]:
```
regression.coef_
```

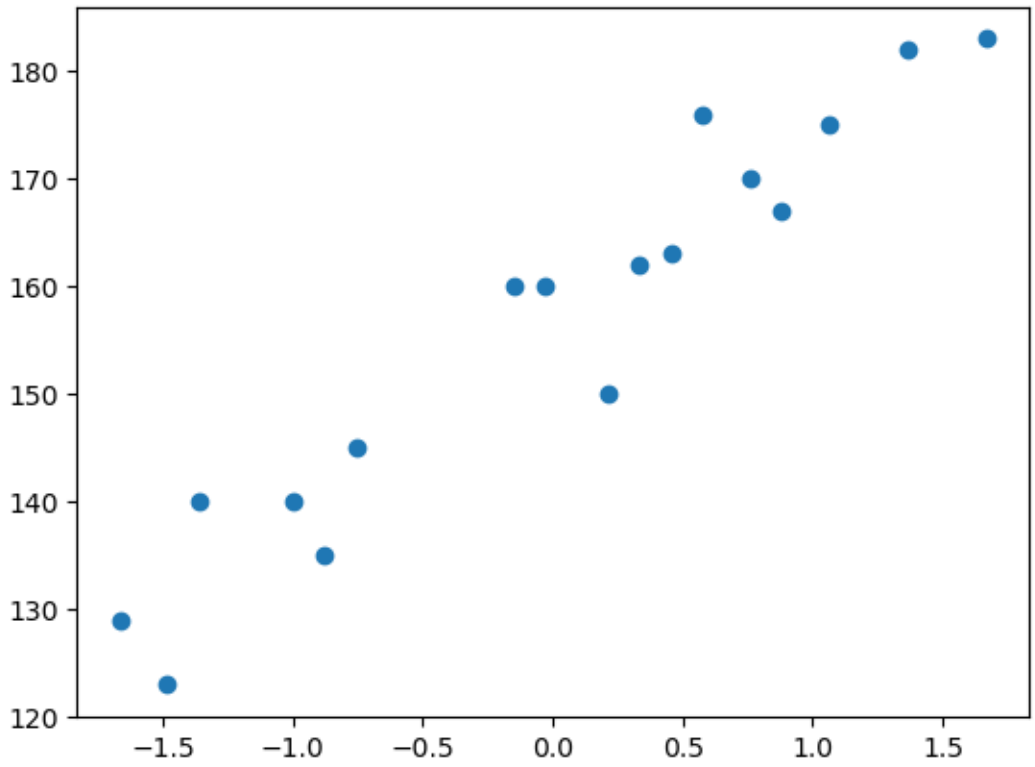
[25]:
```
array([17.2982057])
```

[26]:
```
print("Coefficient or slope: ", regression.coef_)
print("Intercept: ", regression.intercept_)
```

```
Coefficient or slope:  [17.2982057]
Intercept:  156.47058823529412
```

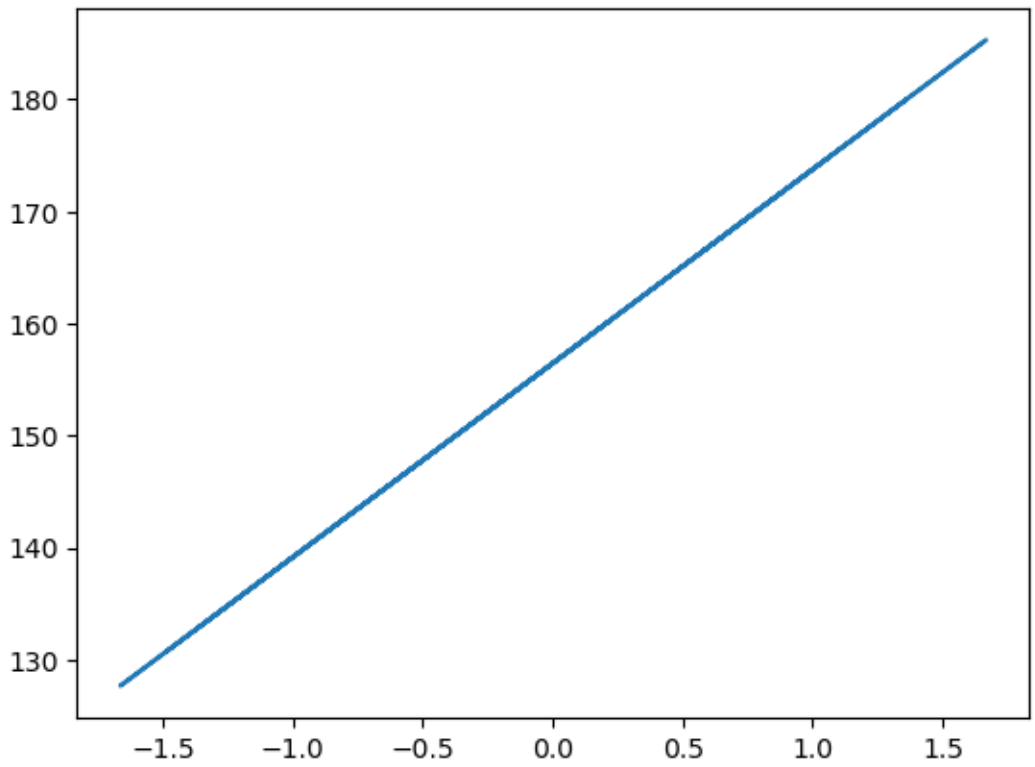
[30]:
```
# Plot training data
plt.scatter(X_train, y_train)
```

[30]: <matplotlib.collections.PathCollection at 0x799511acc6e0>



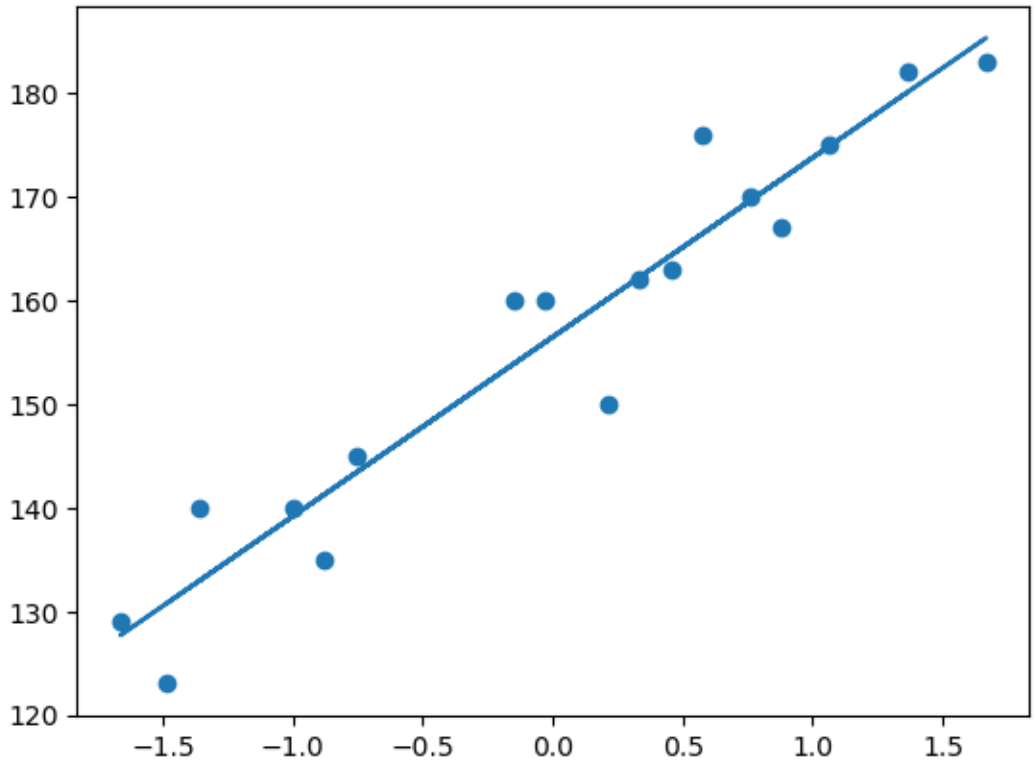
[31]:
```
# best fit line
plt.plot(X_train, regression.predict(X_train))
```

[31]: [<matplotlib.lines.Line2D at 0x79951195f710>]



[32]:
```
plt.scatter(X_train, y_train)
plt.plot(X_train, regression.predict(X_train))
```

[32]: [<matplotlib.lines.Line2D at 0x7995117e9280>]



0.0.3 Prediction of Test Data

1. Predicted height output = intercept + coeff * (Weights)
2. y_pred = 156.47 + 17.29(X_test)

[36]:
```
# Prediction for test data
y_pred = regression.predict(X_test)

y_pred
```

[36]:
```
array([162.26499721, 162.26499721, 127.68347133, 180.07972266,
       148.64197186, 190.55897293])
```

[35]:
```
# Performance metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

[37]:
```
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(mse)
print(mae)
```

```
print(rmse)
```

```
114.84069295228699
9.665125886795005
10.716374991212605
```

0.1 R Square

The formula for R-squared (R^2) is $R^2 = 1$ - (SS_res / SS_tot), where SS_res is the sum of squares of residuals (the differences between observed and predicted values), and SS_tot is the total sum of squares (the differences between observed values and their mean). This formula indicates the proportion of variance in the dependent variable that is predictable from the independent variables.

[38]:
```
from sklearn.metrics import r2_score
```

[41]:
```
score = r2_score(y_test, y_pred)
print(score)
```

```
0.7360826717981276
```

0.2 Adjusted-R Square

The formula for adjusted R-squared is: Adjusted $R^2 = 1$ - [(1 - R^2) * (n - 1) / (n - p - 1)], where R^2 is the regular R-squared, n is the number of observations, and p is the number of predictors. This adjustment accounts for the number of predictors in the model, helping to prevent overfitting.

[43]:
```
1 - (1- score)*(len(y_test)-1)/(len(y_test)-X_test.shape[1]-1)
```

[43]:
```
0.6701033397476595
```

[45]:
```
# OLS Linear Regression
import statsmodels.api as sm
```

[46]:
```
model = sm.OLS(y_train, X_train).fit()
```

[47]:
```
prediction = model.predict(X_test)

print(prediction)
```

```
[  5.79440897   5.79440897 -28.78711691  23.60913442  -7.82861638
  34.08838469]
```

[48]:
```
print(model.summary())
```

```
                                 OLS Regression Results
==============================================================================
=======
Dep. Variable:                 Height   R-squared (uncentered):
0.012
```

```
Model:                            OLS   Adj. R-squared (uncentered):
-0.050
Method:                 Least Squares   F-statistic:
0.1953
Date:                Thu, 04 Jun 2026   Prob (F-statistic):
0.664
Time:                        04:35:20   Log-Likelihood:
-110.03
No. Observations:                  17   AIC:
222.1
Df Residuals:                      16   BIC:
222.9
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
x1            17.2982     39.138      0.442      0.664     -65.671     100.267
==============================================================================
Omnibus:                        0.135   Durbin-Watson:                   0.002
Prob(Omnibus):                  0.935   Jarque-Bera (JB):                0.203
Skew:                          -0.166   Prob(JB):                        0.904
Kurtosis:                       2.581   Cond. No.                         1.00
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not
contain a constant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

[50]:
```
# Prediction for new data
regression.predict(scaler.transform([[72]]))  # We use scaler.transform to␣
 ↪transform the value
```

```
/opt/conda/envs/anaconda-2025.12-py312/lib/python3.12/site-
packages/sklearn/utils/validation.py:2749: UserWarning: X does not have valid
feature names, but StandardScaler was fitted with feature names
  warnings.warn(
```

[50]:
```
array([155.97744705])
```

[ ]:
```

```